

FREE-CODECAMP

DATA-ANALYSIS WITH PYTHON



Challenges ScreenShots

Challenge 1. Mean-StandardVariance Calculator

```
#The values in the returned dictionary should be lists and not Numpy arrays.
def calculate(list):
    try:
        matrix = np.reshape(list,(3,3))
        flattened = matrix.flatten()

        calculations = {
            'mean': [matrix.mean(axis=0), matrix.mean(axis=1), flattened.mean()],
            'var': [matrix.var(axis=0), matrix.var(axis=1), flattened.var()],
            'std': [matrix.std(axis=0), matrix.std(axis=1), flattened.std()],
            'max': [matrix.max(axis=0), matrix.max(axis=1), flattened.max()],
            'min': [matrix.min(axis=0), matrix.min(axis=1), flattened.min()],
            'sum': [matrix.sum(axis=0), matrix.sum(axis=1), flattened.sum()]
        }

        calculations = {
            key: [np.round(val, 2).tolist() for val in values]
            for key, values in calculations.items()
        }

    except ValueError:
        print("List must contain nine numbers.")

    #return calculations

list1 = calculate([55,11,23,32,41,25,63,67,25])
list2 = calculate([10,20,30,40,50,60,70,80,90])
```

```
In [12]: %runfile C:/Users/HP/Documents/3-programacion/code-AI/
Course-DataAnalysis/part4-FinalChallenges/1-mean_var_std.py --wdir
Calculations EXAMPLE 1
mean      [[50.0, 39.67, 24.33], [29.67, 32.67, 51.67], 38.0]
var       [[172.67, 523.56, 0.89], [344.89, 42.89, 358.22], 343.56]
std       [[13.14, 22.88, 0.94], [18.57, 6.55, 18.93], 18.54]
max       [[63, 67, 25], [55, 41, 67], 67]
min       [[32, 11, 23], [11, 25, 25], 11]
sum       [[150, 119, 73], [89, 98, 155], 342]

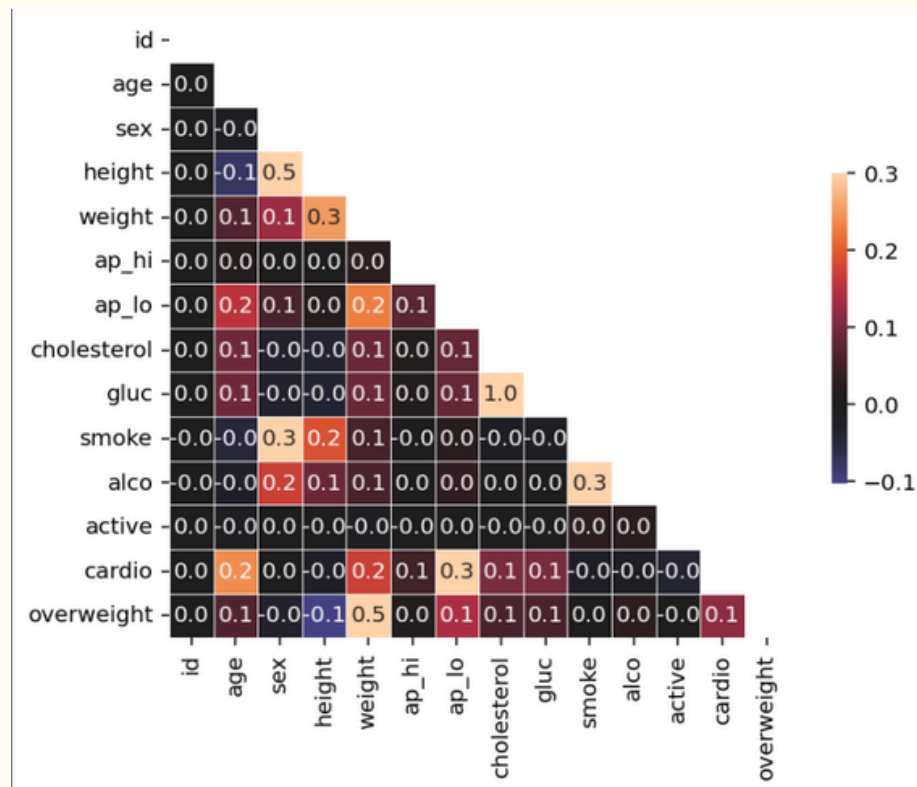
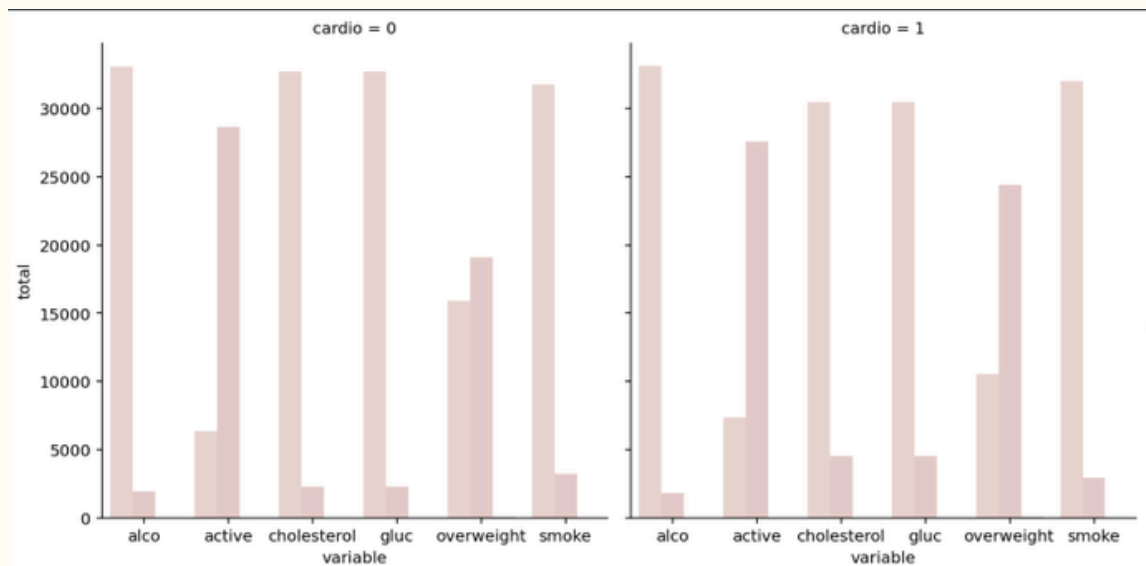
Calculations EXAMPLE 2
mean      [[50.0, 39.67, 24.33], [29.67, 32.67, 51.67], 38.0]
var       [[172.67, 523.56, 0.89], [344.89, 42.89, 358.22], 343.56]
std       [[13.14, 22.88, 0.94], [18.57, 6.55, 18.93], 18.54]
max       [[63, 67, 25], [55, 41, 67], 67]
min       [[32, 11, 23], [11, 25, 25], 11]
sum       [[150, 119, 73], [89, 98, 155], 342]

In [13]:
```

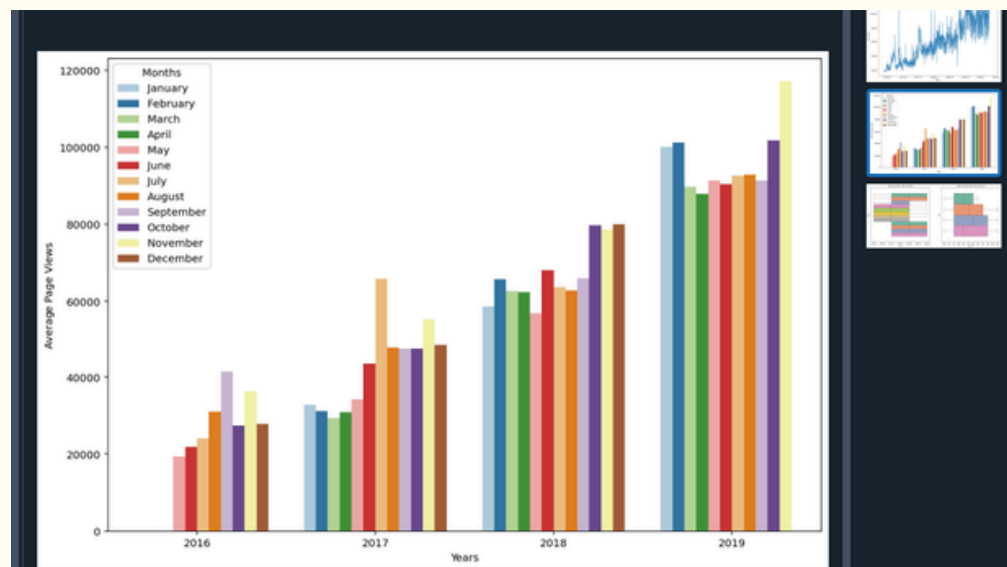
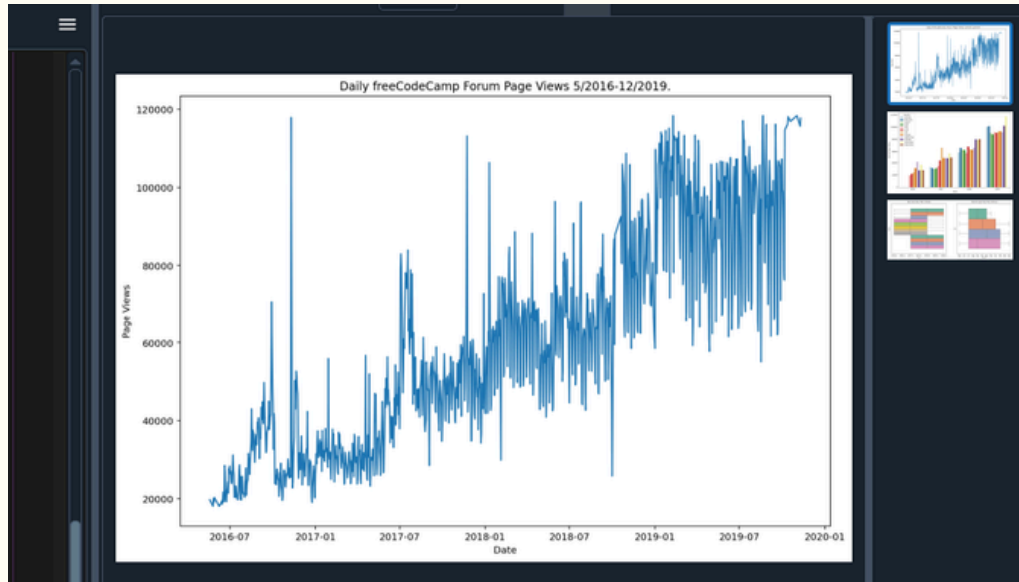
Challenge 2. Demographic Data Analyzer

```
In [13]: %runfile C:/Users/HP/Documents/3-programacion/code-AI/
Course-DataAnalysis/part4-FinalChallenges/2-
demographic_data_analyzer.py --wdir
Number of each race:
  race
White           27816
Black           3124
Asian-Pac-Islander 1039
Amer-Indian-Eskimo 311
Other            271
Name: count, dtype: int64
Average age of men: 39.43354749885268
Percentage with Bachelors degrees: 16.44605509658794%
Percentage with higher education that earn >50K:
46.535843011613935%
Percentage without higher education that earn >50K:
17.3713601914639%
Min work time: 1 hours/week
Percentage of rich among those who work fewest hours: 10.0%
Country with highest percentage of rich: United-States
Highest percentage of rich people in country: 91.45517153424308%
Top occupations in India: 25
```

Challenge 3. Medical Data Visualizer



Challenge 4. FCC Page View Time Series



Challenge 5. Sea Level Predictor

```
Spyder
Archivo  Editar  Buscar  Código fuente  Ejecutar  Depurar  Terminales  Proyectos  Herramientas  Ventana  Ayuda

C:\Users\HP\Documents\3-programacion\code-AI\Cour...ysis\part4-FinalChallenges\sea_level_predictor.py

time_series_visualizer.py x  sea_level_predictor.py x
6  def draw_plot():
7      # Read data from file
8      df = pd.read_csv('epa-sea-level.csv')
9      years = df['Year']
10     sea_level = df['CSIRO Adjusted Sea Level']
11
12     # Create scatter plot
13     fig, ax = plt.subplots(figsize=(14,6))
14     ax = plt.scatter(years, sea_level)
15
16     # Create first line of best fit
17     line1 = linregress(years, sea_level)
18     x1 = np.arange(min(years), 2051)
19     y1 = line1.slope * x1 + line1.intercept
20     plt.plot(x1, y1, color='red', label='Line of Best Fit')
21
22     # Create second line of best fit
23     df2000 = df[df['Year'] >= 2000]
24     line2 = linregress(df2000['Year'], df2000['CSIRO Adjusted Sea Level'])
25     x2 = np.arange(2000, 2051, 1)
26     y2 = line2.slope*x2 + line2.intercept
27     plt.plot(x2, y2)
28
29     # Add labels and title
30     plt.xlabel('Year')
31     plt.ylabel('Sea Level (inches)')
32     plt.title('Rise in Sea Level')
33
34     # Save plot and return data for testing (DO NOT MODIFY)
35     plt.savefig('sea_level_plot.png')
36     return plt.gca()
```

